SysHealth — live demo runbook

Memory pressure across four EC2 instance sizes. Everything below runs from `~/syshealth/terraform`.

1 Fifteen minutes before

Do all of this before anyone is watching. The fleet needs three minutes to boot and calibrate, and you want one rehearsal run in hand.

Bring the fleet up

```
cd ~/syshealth && git pull
./terraform/relaunch.sh
```

Checks your credentials, that the AMI is pinned and the demo tag exists, rewrites `ssh_cidr` to whatever address you are on now, then applies and waits until all four sizes report.

Expect: a table of four sizes, all `online=yes`, then a dashboard URL.

Rehearse once, then let it settle

```
curl -X POST "$(terraform output -raw dashboard_url)/run-stress"
sleep 45 && ./check.sh
```

Expect: micro and small DEGRADED or CRITICAL, medium and large HEALTHY. If not, see section 4 before your teacher arrives.

Then wait about two minutes for pressure to fall back to zero, so the live run starts from a flat line.

2 The demo

Step 1 — Open the dashboard

```
terraform output -raw dashboard_url    # paste into the browser
```

Four cards, all HEALTHY, all green dots, a flat line across the chart.

> **Say:** Four EC2 instances, identical software, different sizes: micro at 1 gigabyte up to large at 8. Each one reads the kernel's own pressure counter every five seconds and reports here. Right now they are all idle.

Step 2 — Explain what is being measured

> **Say:** This is PSI — pressure stall information, straight from the Linux kernel. It is the percentage of time processes were stalled waiting for memory. Not how much memory is used, but how much the machine is struggling to get it. A box can be at 90 percent memory and perfectly happy; this measures the pain, not the level.

Step 3 — Load all four at once

Open **Load test**. Leave it on **Peak load**, 1600 MB for 60 seconds, and point at the preview: the peak is 168% of the micro's RAM and 84% of the small's. Press **Start**. Or from the terminal:

```
curl -X POST "$(terraform output -raw dashboard_url)/run-stress"
```

All four are asked in parallel, within a few milliseconds, so the comparison is fair. Each runs the same job: two workers, 800 MB each, for 60 seconds.

> **Say:** Same workload, same moment, four different machines. Watch which ones feel it.

Step 4 — Watch the separation (60 to 90 seconds)

Instance	RAM	What happens
t3.micro	1 GB	Spikes hard, then the OOM killer ends the job
t3.small	2 GB	Climbs and stays there — the worst sustained case
t3.medium	4 GB	Barely moves
t3.large	8 GB	Flat

> **Say:** The 1.6 gigabyte job fits comfortably on medium and large — they never notice it. On small it fits, but only just, so the kernel spends the whole minute reclaiming memory. That is what pressure looks like.

Step 5 — Show one machine on its own

Click **t3.small** in the Instance toggle. The headline is now one number on a Healthy, Degraded, Critical scale.

> **Say:** That number is the share of the last minute that programs spent waiting for memory. Ten percent means one second in every ten was lost. Under ten is healthy, ten to forty is degraded, forty or more is critical. Those lines are not ours: they come from the systems that act on this signal in production, like Android's low-memory killer, systemd-oomd and Meta's oomd. The sources are in the repo.

Step 6 — The result: which size to buy

Open **Load test** again. **Which size to choose** leads the results: the cheapest size that held the app with room to spare, its price a month, and what the sizes either side would mean.

> **Say:** The answer is t3.medium, about 30 dollars a month. The small would save 15 dollars but it slows down under this load, and the large costs 30 dollars more for room this app does not use. That is a sizing decision made from measurement instead of guesswork, and it is said in money.

To show how much room each size leaves, run a **Variable load** ramp up past the app's need (say 200 to 3200 MB, six steps). The **Pressure at each load level** chart then shows where each size gives out, and the table says how much each holds.

Step 7 — Why, and why not just watch usage

Click **t3.small** while the test runs. Under the tiles, **Where the memory is** splits the RAM, lists what holds it, and names the likely cause.

> **Say:** It does not just say the machine is struggling, it says why. On the small, programs have taken nearly all the RAM, so the kernel is swapping, or throwing out and re-reading the files they run from, and it shows the numbers. And the report checks itself against the usual rule, an alarm at 80 percent memory in use, and says when that rule would have bought the wrong size.

3 Questions you will get

“Why is small worse than micro? Shouldn't the smallest be worst?”

This is the best question anyone can ask you, and you have the evidence.

> **Say:** It does not degrade — it fails. The job needs 1.6 gigabytes; micro has 1 plus half a gigabyte of swap. It does not fit, so the kernel kills the job outright. Pressure collapses because the work stopped, not because the machine coped. Micro's score understates the problem. Small is the worst case you can actually observe: big enough to accept the work, too small to finish it.

Proof, live, if they want it:

```
MICRO=$(terraform output -json instances | python3 -c \
  "import json,sys; print(json.load(sys.stdin)['t3.micro']['public_ip'])")

ssh -i $KEY ubuntu@$MICRO \
  'sudo journalctl -k --since "20 min ago" | grep -i "killed process"'
```

Prints the kernel's own record of killing the stress job, with a timestamp that matches the spike on the chart.

“How is this different from Prometheus and Grafana?”

> **Say:** Prometheus watches. It can graph this same kernel counter, but it never applies a load, never knows what load caused a reading, and never tells you what to buy. SysHealth runs the same load on every size at once, works out how much each can hold before the app slows down, and names the cheapest that fits. They work together, too: SysHealth serves all of it, the verdict included, for Prometheus to scrape.

“Why not just watch memory usage?”

> **Say:** Usage tells you a number, not whether it hurts. A box sitting at 95 percent with nothing to reclaim is fine. A box at 60 percent thrashing is not. PSI measures time lost to waiting, which is the thing users actually feel.

“How do the health states work?”

> **Say:** Fixed thresholds on the kernel's stall measurement, the same on every size, because the number is already a share of time. Under ten percent is healthy, forty or more, or the whole machine frozen ten percent of the time, is critical. They are judged on a one-minute average and must hold for fifteen seconds before the state changes, so one spike is not reported as a failure. We used to compare against each box's idle baseline, but dividing by an idle reading near zero turned a tenth of a percent into a critical alarm.

“What is it built on?”

> **Say:** Python and Flask. The agent reads /proc/pressure/memory and /proc/vmstat. The chart is hand-written SVG, so there is no charting library and nothing loaded from the internet. The whole fleet is Terraform — one command builds all five machines.

“Could this run in production?”

> **Say:** The measurement side, yes, and Prometheus can scrape it today. Two things would need work first: history is kept in memory and is lost if the server restarts, so it would need a real datastore. And the dashboard has no authentication — right now it is locked to a single IP address, which is fine for a demo and not for anything real.

4 If something breaks

The dashboard will not load

Almost always your IP address changed — a different network, or the ISP moved you. The security group still allows the old one. It fails silently and looks exactly like a server that never started.

```
./terraform/relaunch.sh    # detects it, fixes only the firewall rule
```

Expect: `0 to add, 3 to change, 0 to destroy.` No instance may appear in that plan.

A size is missing from the dashboard

```
./check.sh                      # names which sizes are not reporting
terraform output instances      # public IP of each
ssh -i $KEY ubuntu@<ip> 'sudo tail -30 /var/log/syshealth-bootstrap.log'
ssh -i $KEY ubuntu@<ip> 'sudo journalctl -u syshealth -n 30'
```

An agent that is running but cannot reach the dashboard says so in its own log: `push to http://... failed`.

Everything says HEALTHY even under load

The load fits. States are fixed thresholds now, so HEALTHY under load means no program waited long enough to matter. Check the **Right now** tile is moving at all, then raise **Memory to hold** in the Load test form, or run a variable ramp to find where each size gives way.

Flat at 0.00 on every size even while a test runs means stress is not running: `sudo journalctl -u syshealth -n 30` on that box.

Nothing works and you must present anyway

There is a local simulation that needs no AWS and no network. It serves the real dashboard over a simulated four-size fleet, and every page is marked DEMO.

```
cd ~/syshealth
sudo apt install -y python3-flask python3-requests   # first time only
npm run dev        # then open http://127.0.0.1:5000
```

Say plainly that it is simulated. The states and thresholds still come from the same analysis code that runs on the real instances — only the sensor is faked.

5 Afterwards

Save the evidence before you destroy anything. History lives in the server's memory and goes with it. The verdict, as JSON:

```
python3 ../cli.py --server "$(terraform output -raw dashboard_url)" \
  report --save ~/syshealth/report-$(date +%F).json
```

And the raw samples:

```
curl -s "$(terraform output -raw dashboard_url)/series?limit=720" \
  > ~/syshealth/demo-evidence-$(date +%F).json

terraform destroy

aws ec2 describe-instances --region ap-south-1 \
  --filters Name=instance-state-name,Values=running,pending \
  --query 'Reservations[].Instances[].[InstanceId,InstanceType]' --output text
```

Expect: empty output from the last command. That means billing has stopped.

Five instances cost roughly US$5 a day and bill until destroyed.